

Role Overview

This contribution focuses on connecting the admin portal and customer-facing website through shared UI components, global state management, and API service layers. It also covers project documentation and the UI/UX design system.

Responsibilities

- Implemented application-wide state management using Zustand.
- Designed frontend API service layers for communication with the Node.js backend.
- Built reusable components for the admin portal and customer website.
- Supported authentication flow and role-based navigation.
- Organized project documentation and UI/UX design assets.

1. Global State Management

The Zustand stores provide shared state for authentication, order creation, customer carts, and menu data.

File	Purpose	Main Responsibilities
<code>client/src/store/authStore.js</code>	Admin authentication state	Stores the current user and JWT, provides <code>fetchMe()</code> , <code>login()</code> , and <code>logout()</code> , and tracks loading state during authentication checks.
<code>client/src/store/orderStore.js</code>	Admin POS/order state	Manages order items, selected table, and <code>addItem()</code> , <code>removeItem()</code> , <code>updateQuantity()</code> , and <code>clearCart()</code> actions. Cart state is persisted across refreshes.
<code>customer/src/store/cartStore.js</code>	Customer shopping cart	Tracks cart items and quantities, provides <code>addToCart()</code> , <code>removeFromCart()</code> , <code>updateQty()</code> ,

and `clearCart()`, and derives the cart total and item count.

<code>customer/src/store/menuStore.js</code>	Customer menu data	Stores menu categories and items fetched from the API, tracks loading and error states, and shares menu data between menu and cart components.
--	--------------------	--

Why This Matters

Centralized state helps different pages and components use consistent data without passing every value through multiple layers of components.

2. Frontend API Service Layer

The API service files centralize requests from the React applications to the Node.js backend.

Admin API

File: `client/src/services/api.js`

Responsibilities:

- Creates an Axios instance configured with the backend base URL.
- Attaches the JWT to requests through an Axios request interceptor.
- Provides functions for orders, menu items, tables, reservations, analytics, dashboard statistics, staff, and billing.
- Handles unauthorized responses (401) and redirects to the login page.

Functions documented for this service include:

- **Orders:** `getOrders()`, `createOrder()`, `updateOrderStatus()`
- **Menu:** `getMenuItems()`, `createMenuItem()`, `updateMenuItem()`
- **Tables:** `getTables()`, `updateTableStatus()`
- **Reservations:** `getReservations()`, `createReservation()`
- **Analytics:** `getAnalytics()`, `getDashboardStats()`
- **Staff:** `getStaff()`, `createStaff()`, `deleteStaff()`
- **Billing:** `getBillingOrders()`, `createPayment()`

Customer API

File: `customer/src/services/api.js`

Responsibilities:

- Uses the public API base URL.
- Provides functions for customer-facing menu and ordering features.
- Centralizes requests for reservations, reviews, and order tracking.

Functions documented for this service include:

- `getPublicMenu()` — Retrieves available menu items.
- `getCategories()` — Retrieves menu categories.
- `placeOrder()` — Submits a customer order.
- `createReservation()` — Submits a table reservation.
- `submitReview()` — Submits a customer review.
- `trackOrder(orderId)` — Retrieves an order's status.

3. Reusable UI Components

Reusable components provide consistent navigation and layout across the admin portal and customer website.

Admin Portal Components

File	Purpose
<code>client/src/components/Layout.jsx</code>	Wraps admin pages in a shared responsive layout containing the sidebar and topbar. Manages the mobile sidebar state, page-content spacing, and theme tokens.
<code>client/src/components/Sidebar.jsx</code>	Displays navigation links, highlights the active route using React Router, shows role-specific navigation, displays user information, and provides logout functionality.
<code>client/src/components/Topbar.jsx</code>	Displays the current page title, date and time, avatar, and profile information.
<code>client/src/pages/Auth/Login.jsx</code>	Provides the staff login form, validates input, calls the authentication service, stores the JWT through <code>authStore</code> , and redirects users after successful login.

Customer Website Components

File	Purpose
<code>customer/src/components/Navbar.jsx</code>	Displays the 22nd Parallel brand, Home and Menu links, and a live cart-item count. Supports sticky positioning and smooth scrolling.
<code>customer/src/components/CartDrawer.jsx</code>	Displays cart items, quantities, prices, and the running total. Supports quantity updates, item removal, and order placement through the API service.

4. Documentation and Design Assets

File or Directory	Purpose
<code>README.md</code>	Main GitHub documentation, including the project overview, technology stack, setup instructions, demo access details, feature list, and menu-data attribution.
<code>CONTRIBUTORS.md</code>	Documents team members and their respective project contributions.
<code>open-design/</code>	Contains UI/UX design assets, system-design documentation, component specifications, and design-system tokens.

5. Suggested Git Commit Plan

Use these commits if the listed files are part of your local project and are ready to commit.

Commit 1 — Global State Management

Files:

- `client/src/store/authStore.js`
- `client/src/store/orderStore.js`
- `customer/src/store/cartStore.js`
- `customer/src/store/menuStore.js`

Commit message:

feat: add Zustand stores for authentication, orders, cart, and menu

Commit 2 — API Service Layer

Files:

- `client/src/services/api.js`
- `customer/src/services/api.js`

Commit message:

feat: add API service layers for admin and customer applications

Commit 3 — Admin UI Components

Files:

- `client/src/components/Layout.jsx`
- `client/src/components/Sidebar.jsx`
- `client/src/components/Topbar.jsx`
- `client/src/pages/Auth/Login.jsx`

Commit message:

feat: add shared admin layout, navigation, topbar, and login

Commit 4 — Customer UI Components

Files:

- `customer/src/components/Navbar.jsx`
- `customer/src/components/CartDrawer.jsx`

Commit message:

feat: add customer navigation and shopping cart drawer

Commit 5 — Documentation and Design Assets

Files:

- `README.md`
- `CONTRIBUTORS.md`
- `open-design/`

Commit message:

docs: update project documentation and UI/UX design assets

6. Before Publishing to GitHub

- Confirm that the files listed in this document exist in the repository.
- Make sure `README.md` includes the actual installation and run commands for your project.
- Do not commit real passwords, private API keys, JWTs, or production credentials.
- If demo credentials are required, use safe, project-specific test credentials and explain that they are for local development only.
- Check that `.env` files and other secrets are excluded through `.gitignore`.
- Verify that the `open-design/` directory contains only assets you are allowed to publish.

Contribution Summary

This work focuses on the frontend architecture of the 22nd Parallel restaurant application: shared Zustand state, Axios-based API integration, reusable admin and customer components, authentication-related UI, and project documentation/design assets.